Communication

Date	8 July 2026
Team ID	XXXXXX
Project Name	Smart Lender
Maximum Marks	1 Mark

Communication Plan:

Effective communication is essential for a successful project demonstration. This document outlines the communication strategy used within the team and with stakeholders throughout the project lifecycle, including how updates, issues, and feedback were managed.

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	WhatsApp / Slack	All Team Members (Tejasri, Hima, Raghava, Haneef, Triveni)	Sync on daily progress, discuss what tasks are completed from the 22 total tasks, and identify immediate blockers.
2	Progress Update	Weekly	GitHub / Google Meet	All Team Members	Review epic and story completion status, update the project repository, and plan the upcoming week's milestones.

S.No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
3	Issue / Bug Discussion	As Needed	Discord / Teams	Relevant Developers	Debug algorithm performance issues (e.g., tuning Random Forest, KNN, or optimizing the XGBoost model to close the training/testing accuracy gap).
4	Stakeholder Review	Bi-Weekly	Email / SmartBridge Platform	Team Lead (Tejasri) & Platform Evaluators	Submit progress updates, review model accuracy reports, and ensure the project remains aligned with evaluation criteria.
5	Final Demo Rehearsal	Once	Google Meet / Zoom	All Team Members	Run a complete end-to-end trial of the Flask web application deployed on IBM Cloud, testing both low-risk and high-risk applicant scenarios.

Communication Challenges & Resolutions:

S.No	Challenge Faced	Resolution / Action Taken
1	Inconsistencies in training and testing data splits among team members while evaluating different models (KNN, Decision Tree, Random Forest, XGBoost).	Standardized the preprocessing pipeline by centralizing the data cleaning script (handling mean/mode missing values) in a shared GitHub branch to ensure all members evaluated models on the exact same dataset.
2	Tracking individual progress across 9 Epics and 22 Tasks led to confusion regarding who was responsible for specific tasks.	Assigned clear roles using a shared project tracking board, dividing responsibilities among the five members (e.g., separating Flask UI design from backend ML model training).
3	Resolving dependency and environment mismatches (e.g., mismatched Scikit-learn or XGBoost library versions) when deploying the trained model into the Flask app.	Created a shared requirements.txt file and set up identical Python environment configurations using Anaconda Navigator to ensure seamless integration of the model into the Flask web framework.